```
Mosquito Species (With Respective Common Disease) Classification

This project aims to develop an accurate and EfficientNetV2B0-based
Convolutional Neural Network (CNN)
that will detect three (3) classifications of mosquito species (I
added additional information
about the common disseases these species commonly transmit).

The dataset used in this project is sourced from Kaggle:
    Mosquito dataset for classification cnn
    (https://www.kaggle.com/datasets/masud1901/mosquito-dataset-for-
classification-cnn/data)
which is a meticulously curated collection of high-quality images
featuring three major mosquito species:
    Culex, Anopheles, and Aedes.
With 3,000 images in total, the dataset was structured to ensure a
balanced representation of the three species,
    each having 1,000 images.

In this project, the dataset was split into:
    70% Training
    15% Validation
    15% Testing

After the training and fine-tuning, the model achieved a test accuracy
of 100%.



Configure Kaggle API Credentials Path

import os
os.environ['KAGGLE_CONFIG_DIR'] = r'C:\Users\user\.kaggle'

Install and Test Kaggle CLI

!pip install -q kaggle
!kaggle --version
```

```
[notice] A new release of pip is available: 25.0.1 -> 25.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
Kaggle API 1.7.4.5
```

```
Download and Unzip Mosquito Dataset from Kaggle

!kaggle datasets download masud1901/mosquito-dataset-for-
classification-cnn --unzip -p mosquito_data
```

```
Dataset URL: https://www.kaggle.com/datasets/masud1901/mosquito-
dataset-for-classification-cnn
```

```
License(s): apache-2.0
Downloading mosquito-dataset-for-classification-cnn.zip to
mosquito_data



  0%|          | 0.00/54.2M [00:00<?, ?B/s]
 61%|######    | 33.0M/54.2M [00:00<00:00, 342MB/s]
100%|##########| 54.2M/54.2M [00:00<00:00, 431MB/s]
```

```
Import Required Libraries
    This part defines the is_image_okay function, which checks if the
file at img_path
        - is a valid, readable image.
        - Converts the image to RGB if possible.
        - Returns True if the image is good, and False if it’s
corrupted, unreadable, or not an image.
```

```
import os, shutil, random
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.applications import EfficientNetV2B0
import seaborn as sns
from PIL import Image
from PIL import UnidentifiedImageError

def is_image_okay(img_path):
    try:
        img = Image.open(img_path)
        img.verify()  # Checks if it's a readable image file
        img = Image.open(img_path)  # Re-open for reading after
verify()
        img = img.convert("RGB")    # Ensures it's RGB for model
        return True
    except (UnidentifiedImageError, OSError, ValueError):
        return False
```

```
Load Image Datasets and Explore Class Balance
```

```
DATASET_PATH = 'mosquito_data/Mosquito_dataset'
print("Species folders:", os.listdir(DATASET_PATH))
for sp in os.listdir(DATASET_PATH):
    folder = os.path.join(DATASET_PATH, sp)
    print(f"{sp}: {len(os.listdir(folder))} files")
    for fname in os.listdir(folder):
        if not fname.lower().endswith(('jpg', 'jpeg', 'png')):
            os.remove(os.path.join(folder, fname))
```

```
Species folders: ['AEDES', 'ANOPHELES', 'CULEX']
AEDES: 1000 files
ANOPHELES: 1000 files
CULEX: 1000 files
```

Remove Corrupted or Unreadable Images

```
for sp in os.listdir(DATASET_PATH):
    folder = os.path.join(DATASET_PATH, sp)
    for fname in os.listdir(folder):
        img_path = os.path.join(folder, fname)
        if not is_image_okay(img_path):
            os.remove(img_path)
            print("Removed:", img_path)
```

Split Dataset into Train, Val, Test (70/15/15)

```
base_dir = 'mosquito_split'
train_dir = os.path.join(base_dir, 'train')
val_dir = os.path.join(base_dir, 'val')
test_dir = os.path.join(base_dir, 'test')

def split_data(SOURCE, TRAINING, VALIDATION, TESTING,
split_ratio=(0.7, 0.15, 0.15), seed=42):
    random.seed(seed)
    for category in os.listdir(SOURCE):
        files = os.listdir(os.path.join(SOURCE, category))
        files = [f for f in files if f.lower().endswith(('jpg',
'jpeg', 'png'))]
        random.shuffle(files)
        n = len(files)
        n_train = int(n * split_ratio[0])
        n_val = int(n * split_ratio[1])
        splits = [TRAINING, VALIDATION, TESTING]
        indices = [(0, n_train), (n_train, n_train + n_val), (n_train
+ n_val, n)]
        for (split_folder, (start, end)) in zip(splits, indices):
            cat_folder = os.path.join(split_folder, category)
            os.makedirs(cat_folder, exist_ok=True)
            for fname in files[start:end]:
                shutil.copy(os.path.join(SOURCE, category, fname),
os.path.join(cat_folder, fname))

if not os.path.exists(base_dir):
    os.makedirs(base_dir, exist_ok=True)
    split_data(DATASET_PATH, train_dir, val_dir, test_dir)
```

Load Image Datasets and Explore Class Balance (Again)

```
IMG_SIZE = 224
BATCH_SIZE = 32
```

```
train_ds = keras.utils.image_dataset_from_directory(
    train_dir, image_size=(IMG_SIZE, IMG_SIZE), batch_size=BATCH_SIZE,
shuffle=True, seed=42
)
val_ds = keras.utils.image_dataset_from_directory(
    val_dir, image_size=(IMG_SIZE, IMG_SIZE), batch_size=BATCH_SIZE,
shuffle=True, seed=42
)
test_ds = keras.utils.image_dataset_from_directory(
    test_dir, image_size=(IMG_SIZE, IMG_SIZE), batch_size=BATCH_SIZE,
shuffle=False
)

class_names = train_ds.class_names
print("Class labels:", class_names)
```

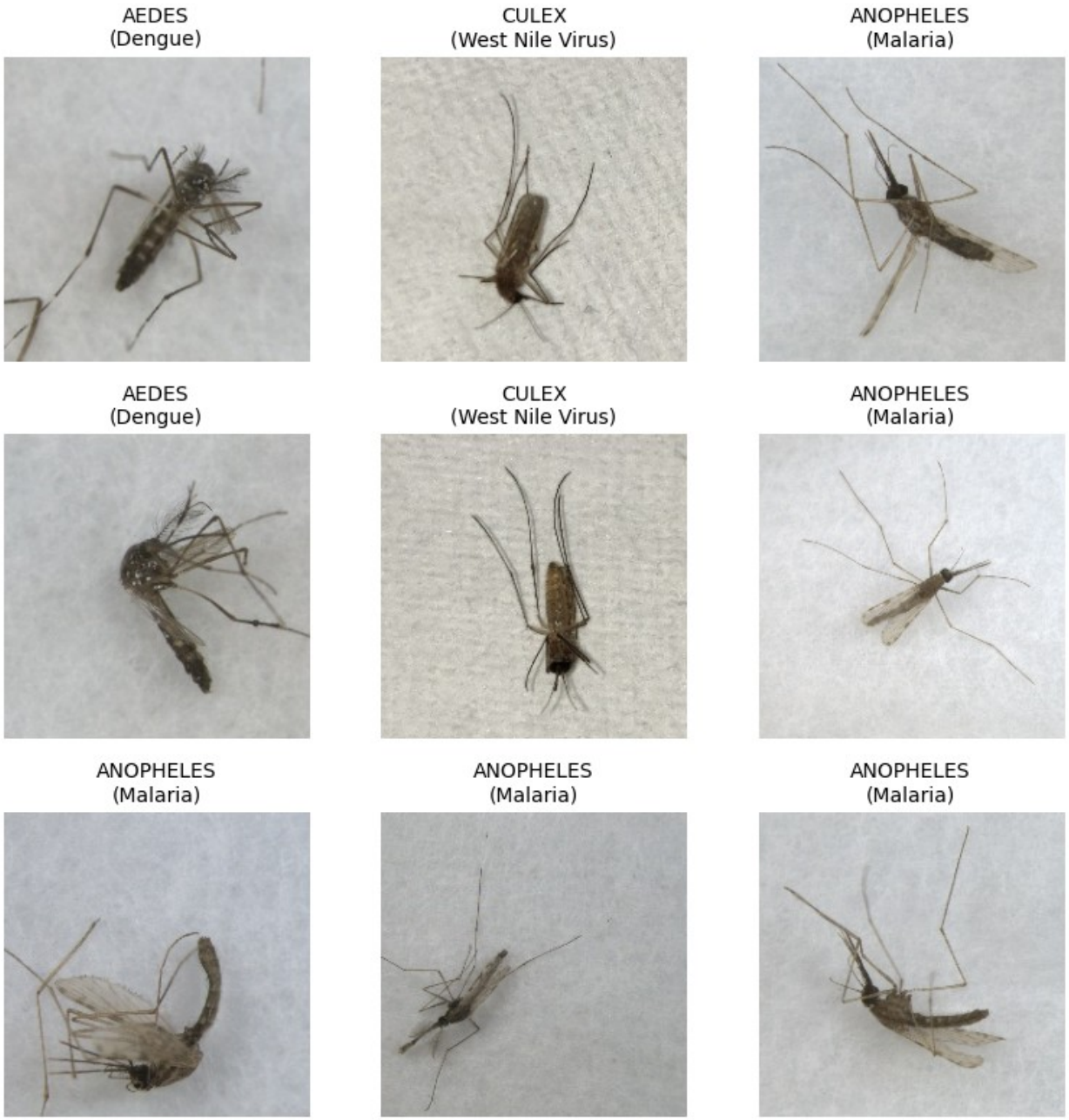
```
Found 2100 files belonging to 3 classes.
Found 450 files belonging to 3 classes.
Found 450 files belonging to 3 classes.
Class labels: ['AEDES', 'ANOPHELES', 'CULEX']
```

Mapping Mosquito Species to Diseases Transmitted

```
species_to_disease = {
    'AEDES': 'Dengue',
    'ANOPHELES': 'Malaria',
    'CULEX': 'West Nile Virus'
}
```

Visualize Example Images from Train Set

```
plt.figure(figsize=(8, 8))
for images, labels in train_ds.take(1):
    for i in range(9):  # 9 images for a 3x3 grid
        plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        species = class_names[labels[i]]
        disease = species_to_disease.get(species, "Unknown")
        plt.title(f"{species}\n({disease})", fontsize=10)
        plt.axis("off")
plt.tight_layout()
plt.show()
```




```
Data Augmentation Pipeline

data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.15),
    layers.RandomZoom(0.2),
    layers.RandomContrast(0.2),
])

Build EfficientNetV2B0-based Model with Transfer Learning
```

```
base_model = EfficientNetV2B0(
    include_top=False,
    input_shape=(IMG_SIZE, IMG_SIZE, 3),
    weights="imagenet"
)
base_model.trainable = False

inputs = keras.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
x = data_augmentation(inputs)
x = tf.keras.applications.efficientnet_v2.preprocess_input(x)
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.BatchNormalization()(x)
x = layers.Dropout(0.4)(x)
x = layers.Dense(128, activation='relu')(x)
x = layers.BatchNormalization()(x)
outputs = layers.Dense(len(class_names), activation='softmax')(x)
model = keras.Model(inputs, outputs)
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-4),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
model.summary()
```

```
Model: "functional_5"

┏━━━
```

```
──────────────────┤
│ batch_normalization_4                │ (None, 1280)                │
5,120 │
│ (BatchNormalization)                 │                             │
│
├──────────────────────────────────────┼─────────────────────────────┼
──────────────────┤
│ dropout_2 (Dropout)                  │ (None, 1280)                │
0 │
├──────────────────────────────────────┼─────────────────────────────┼
──────────────────┤
│ dense_4 (Dense)                      │ (None, 128)                 │
163,968 │
├──────────────────────────────────────┼─────────────────────────────┼
──────────────────┤
│ batch_normalization_5                │ (None, 128)                 │
512 │
│ (BatchNormalization)                 │                             │
│
├──────────────────────────────────────┼─────────────────────────────┼
──────────────────┤
│ dense_5 (Dense)                      │ (None, 3)                   │
387 │
└──────────────────────────────────────┴─────────────────────────────┴
──────────────────┘

 Total params: 6,089,299 (23.23 MB)

 Trainable params: 167,171 (653.01 KB)

 Non-trainable params: 5,922,128 (22.59 MB)
```

```
Training the Model with EarlyStopping and Checkpoints

callbacks = [
    keras.callbacks.EarlyStopping(monitor='val_loss', patience=6,
restore_best_weights=True),
    keras.callbacks.ModelCheckpoint('best_mosquito_model.keras',
monitor='val_accuracy', save_best_only=True)
]
history = model.fit(
    train_ds.prefetch(tf.data.AUTOTUNE),
    validation_data=val_ds.prefetch(tf.data.AUTOTUNE),
    epochs=50,
    callbacks=callbacks
)
```

```
Epoch 1/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 168s 2s/step - accuracy: 0.5179 - loss:
1.1618 - val_accuracy: 0.8844 - val_loss: 0.6543
Epoch 2/50
```

```
66/66 ━━━━━━━━━━━━━━━━━━━━ 162s 2s/step - accuracy: 0.8826 - loss:
0.3096 - val_accuracy: 0.9644 - val_loss: 0.4116
Epoch 3/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 165s 3s/step - accuracy: 0.9191 - loss:
0.2156 - val_accuracy: 0.9800 - val_loss: 0.2507
Epoch 4/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 353s 5s/step - accuracy: 0.9527 - loss:
0.1525 - val_accuracy: 0.9867 - val_loss: 0.1485
Epoch 5/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 64s 964ms/step - accuracy: 0.9504 - loss:
0.1437 - val_accuracy: 0.9889 - val_loss: 0.0888
Epoch 6/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 110s 2s/step - accuracy: 0.9572 - loss:
0.1305 - val_accuracy: 0.9933 - val_loss: 0.0561
Epoch 7/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 150s 2s/step - accuracy: 0.9605 - loss:
0.1156 - val_accuracy: 0.9956 - val_loss: 0.0385
Epoch 8/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 160s 2s/step - accuracy: 0.9648 - loss:
0.1022 - val_accuracy: 0.9956 - val_loss: 0.0285
Epoch 9/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 191s 2s/step - accuracy: 0.9806 - loss:
0.0712 - val_accuracy: 0.9956 - val_loss: 0.0226
Epoch 10/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 181s 2s/step - accuracy: 0.9714 - loss:
0.0996 - val_accuracy: 0.9978 - val_loss: 0.0176
Epoch 11/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 140s 2s/step - accuracy: 0.9718 - loss:
0.0798 - val_accuracy: 1.0000 - val_loss: 0.0144
Epoch 12/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 143s 2s/step - accuracy: 0.9674 - loss:
0.0804 - val_accuracy: 1.0000 - val_loss: 0.0121
Epoch 13/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 664s 9s/step - accuracy: 0.9781 - loss:
0.0657 - val_accuracy: 1.0000 - val_loss: 0.0100
Epoch 14/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 69s 1s/step - accuracy: 0.9749 - loss:
0.0702 - val_accuracy: 1.0000 - val_loss: 0.0086
Epoch 15/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 139s 2s/step - accuracy: 0.9743 - loss:
0.0686 - val_accuracy: 1.0000 - val_loss: 0.0082
Epoch 16/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 127s 2s/step - accuracy: 0.9779 - loss:
0.0652 - val_accuracy: 1.0000 - val_loss: 0.0076
Epoch 17/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 159s 2s/step - accuracy: 0.9784 - loss:
0.0603 - val_accuracy: 1.0000 - val_loss: 0.0065
Epoch 18/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 166s 3s/step - accuracy: 0.9782 - loss:
```

```
0.0629 - val_accuracy: 1.0000 - val_loss: 0.0060
Epoch 19/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 140s 2s/step - accuracy: 0.9732 - loss:
0.0833 - val_accuracy: 1.0000 - val_loss: 0.0054
Epoch 20/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 129s 2s/step - accuracy: 0.9817 - loss:
0.0505 - val_accuracy: 1.0000 - val_loss: 0.0049
Epoch 21/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 131s 2s/step - accuracy: 0.9867 - loss:
0.0482 - val_accuracy: 1.0000 - val_loss: 0.0045
Epoch 22/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 124s 2s/step - accuracy: 0.9796 - loss:
0.0578 - val_accuracy: 1.0000 - val_loss: 0.0044
Epoch 23/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 91s 1s/step - accuracy: 0.9898 - loss:
0.0375 - val_accuracy: 1.0000 - val_loss: 0.0043
Epoch 24/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 73s 1s/step - accuracy: 0.9787 - loss:
0.0469 - val_accuracy: 1.0000 - val_loss: 0.0040
Epoch 25/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 93s 1s/step - accuracy: 0.9833 - loss:
0.0452 - val_accuracy: 1.0000 - val_loss: 0.0039
Epoch 26/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 75s 1s/step - accuracy: 0.9872 - loss:
0.0425 - val_accuracy: 1.0000 - val_loss: 0.0034
Epoch 27/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 89s 1s/step - accuracy: 0.9835 - loss:
0.0517 - val_accuracy: 1.0000 - val_loss: 0.0031
Epoch 28/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 81s 1s/step - accuracy: 0.9900 - loss:
0.0402 - val_accuracy: 1.0000 - val_loss: 0.0031
Epoch 29/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 81s 1s/step - accuracy: 0.9844 - loss:
0.0420 - val_accuracy: 1.0000 - val_loss: 0.0031
Epoch 30/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 78s 1s/step - accuracy: 0.9857 - loss:
0.0467 - val_accuracy: 1.0000 - val_loss: 0.0028
Epoch 31/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 80s 1s/step - accuracy: 0.9887 - loss:
0.0339 - val_accuracy: 1.0000 - val_loss: 0.0023
Epoch 32/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 95s 1s/step - accuracy: 0.9848 - loss:
0.0463 - val_accuracy: 1.0000 - val_loss: 0.0023
Epoch 33/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 89s 1s/step - accuracy: 0.9853 - loss:
0.0405 - val_accuracy: 1.0000 - val_loss: 0.0021
Epoch 34/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 76s 1s/step - accuracy: 0.9870 - loss:
0.0417 - val_accuracy: 1.0000 - val_loss: 0.0024
```

```
Epoch 35/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 99s 2s/step - accuracy: 0.9852 - loss:
0.0376 - val_accuracy: 1.0000 - val_loss: 0.0022
Epoch 36/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 105s 2s/step - accuracy: 0.9876 - loss:
0.0380 - val_accuracy: 1.0000 - val_loss: 0.0020
Epoch 37/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 125s 1s/step - accuracy: 0.9907 - loss:
0.0337 - val_accuracy: 1.0000 - val_loss: 0.0017
Epoch 38/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 148s 1s/step - accuracy: 0.9923 - loss:
0.0272 - val_accuracy: 1.0000 - val_loss: 0.0018
Epoch 39/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 77s 1s/step - accuracy: 0.9874 - loss:
0.0338 - val_accuracy: 1.0000 - val_loss: 0.0016
Epoch 40/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 89s 1s/step - accuracy: 0.9848 - loss:
0.0400 - val_accuracy: 1.0000 - val_loss: 0.0016
Epoch 41/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 78s 1s/step - accuracy: 0.9893 - loss:
0.0334 - val_accuracy: 1.0000 - val_loss: 0.0017
Epoch 42/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 77s 1s/step - accuracy: 0.9896 - loss:
0.0365 - val_accuracy: 1.0000 - val_loss: 0.0015
Epoch 43/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 76s 1s/step - accuracy: 0.9906 - loss:
0.0288 - val_accuracy: 1.0000 - val_loss: 0.0015
Epoch 44/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 81s 1s/step - accuracy: 0.9849 - loss:
0.0365 - val_accuracy: 1.0000 - val_loss: 0.0014
Epoch 45/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 77s 1s/step - accuracy: 0.9895 - loss:
0.0299 - val_accuracy: 1.0000 - val_loss: 0.0013
Epoch 46/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 76s 1s/step - accuracy: 0.9931 - loss:
0.0230 - val_accuracy: 1.0000 - val_loss: 0.0013
Epoch 47/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 77s 1s/step - accuracy: 0.9871 - loss:
0.0366 - val_accuracy: 1.0000 - val_loss: 0.0013
Epoch 48/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 83s 1s/step - accuracy: 0.9870 - loss:
0.0332 - val_accuracy: 1.0000 - val_loss: 0.0013
Epoch 49/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 80s 1s/step - accuracy: 0.9835 - loss:
0.0450 - val_accuracy: 1.0000 - val_loss: 0.0013
Epoch 50/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 88s 1s/step - accuracy: 0.9853 - loss:
0.0366 - val_accuracy: 1.0000 - val_loss: 0.0011
```

Fine-Tune the Model

```
base_model.trainable = True
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-5),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
finetune_history = model.fit(
    train_ds.prefetch(tf.data.AUTOTUNE),
    validation_data=val_ds.prefetch(tf.data.AUTOTUNE),
    epochs=20,
    callbacks=callbacks
)
```

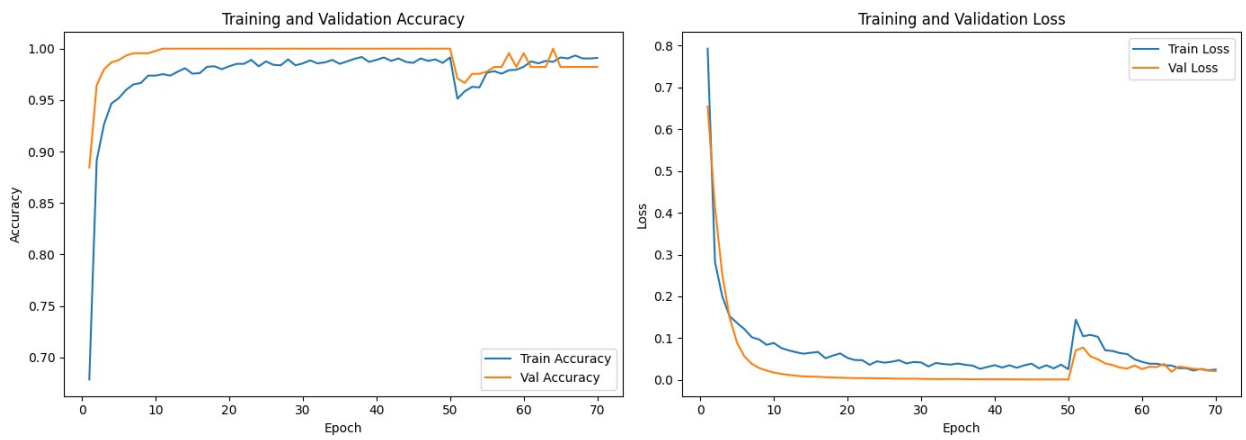
```
Epoch 1/20
66/66 ━━━━━━━━━━━━━━━━━━━━ 546s 7s/step - accuracy: 0.9487 - loss:
0.1567 - val_accuracy: 0.9711 - val_loss: 0.0708
Epoch 2/20
66/66 ━━━━━━━━━━━━━━━━━━━━ 487s 7s/step - accuracy: 0.9623 - loss:
0.0915 - val_accuracy: 0.9667 - val_loss: 0.0773
Epoch 3/20
66/66 ━━━━━━━━━━━━━━━━━━━━ 492s 7s/step - accuracy: 0.9572 - loss:
0.1203 - val_accuracy: 0.9756 - val_loss: 0.0565
Epoch 4/20
66/66 ━━━━━━━━━━━━━━━━━━━━ 478s 7s/step - accuracy: 0.9596 - loss:
0.1177 - val_accuracy: 0.9756 - val_loss: 0.0495
Epoch 5/20
66/66 ━━━━━━━━━━━━━━━━━━━━ 502s 8s/step - accuracy: 0.9673 - loss:
0.0884 - val_accuracy: 0.9778 - val_loss: 0.0395
Epoch 6/20
66/66 ━━━━━━━━━━━━━━━━━━━━ 565s 8s/step - accuracy: 0.9769 - loss:
0.0720 - val_accuracy: 0.9822 - val_loss: 0.0357
Epoch 7/20
66/66 ━━━━━━━━━━━━━━━━━━━━ 299s 4s/step - accuracy: 0.9770 - loss:
0.0680 - val_accuracy: 0.9822 - val_loss: 0.0297
Epoch 8/20
66/66 ━━━━━━━━━━━━━━━━━━━━ 300s 4s/step - accuracy: 0.9807 - loss:
0.0578 - val_accuracy: 0.9956 - val_loss: 0.0274
Epoch 9/20
66/66 ━━━━━━━━━━━━━━━━━━━━ 274s 4s/step - accuracy: 0.9790 - loss:
0.0488 - val_accuracy: 0.9822 - val_loss: 0.0345
Epoch 10/20
66/66 ━━━━━━━━━━━━━━━━━━━━ 322s 4s/step - accuracy: 0.9809 - loss:
0.0473 - val_accuracy: 0.9956 - val_loss: 0.0259
Epoch 11/20
66/66 ━━━━━━━━━━━━━━━━━━━━ 261s 4s/step - accuracy: 0.9867 - loss:
0.0389 - val_accuracy: 0.9822 - val_loss: 0.0316
Epoch 12/20
66/66 ━━━━━━━━━━━━━━━━━━━━ 273s 4s/step - accuracy: 0.9867 - loss:
0.0418 - val_accuracy: 0.9822 - val_loss: 0.0311
Epoch 13/20
```

```
66/66 ━━━━━━━━━━━━━━━━━━━━ 275s 4s/step - accuracy: 0.9828 - loss:
0.0443 - val_accuracy: 0.9822 - val_loss: 0.0382
Epoch 14/20
66/66 ━━━━━━━━━━━━━━━━━━━━ 271s 4s/step - accuracy: 0.9885 - loss:
0.0343 - val_accuracy: 1.0000 - val_loss: 0.0197
Epoch 15/20
66/66 ━━━━━━━━━━━━━━━━━━━━ 286s 4s/step - accuracy: 0.9921 - loss:
0.0235 - val_accuracy: 0.9822 - val_loss: 0.0319
Epoch 16/20
66/66 ━━━━━━━━━━━━━━━━━━━━ 343s 5s/step - accuracy: 0.9872 - loss:
0.0355 - val_accuracy: 0.9822 - val_loss: 0.0297
Epoch 17/20
66/66 ━━━━━━━━━━━━━━━━━━━━ 270s 4s/step - accuracy: 0.9906 - loss:
0.0259 - val_accuracy: 0.9822 - val_loss: 0.0266
Epoch 18/20
66/66 ━━━━━━━━━━━━━━━━━━━━ 262s 4s/step - accuracy: 0.9904 - loss:
0.0290 - val_accuracy: 0.9822 - val_loss: 0.0257
Epoch 19/20
66/66 ━━━━━━━━━━━━━━━━━━━━ 333s 4s/step - accuracy: 0.9913 - loss:
0.0243 - val_accuracy: 0.9822 - val_loss: 0.0223
Epoch 20/20
66/66 ━━━━━━━━━━━━━━━━━━━━ 269s 4s/step - accuracy: 0.9909 - loss:
0.0243 - val_accuracy: 0.9822 - val_loss: 0.0213
```

Plot Learning Curve (Training & Validation Accuracy and Loss)

```
def plot_learning_curves(history, finetune_history=None):
    acc = history.history['accuracy']
    val_acc = history.history['val_accuracy']
    loss = history.history['loss']
    val_loss = history.history['val_loss']

    if finetune_history is not None:
        acc += finetune_history.history['accuracy']
        val_acc += finetune_history.history['val_accuracy']
        loss += finetune_history.history['loss']
        val_loss += finetune_history.history['val_loss']

    epochs_range = range(1, len(acc) + 1)

    plt.figure(figsize=(14, 5))
    # Accuracy
    plt.subplot(1, 2, 1)
    plt.plot(epochs_range, acc, label='Train Accuracy')
    plt.plot(epochs_range, val_acc, label='Val Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.title('Training and Validation Accuracy')
    plt.legend()
```

```
    # Loss
    plt.subplot(1, 2, 2)
    plt.plot(epochs_range, loss, label='Train Loss')
    plt.plot(epochs_range, val_loss, label='Val Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Training and Validation Loss')
    plt.legend()

    plt.tight_layout()
    plt.show()

plot_learning_curves(history, finetune_history)
```




```
Evaluate on Test Set

model.load_weights('best_mosquito_model.keras')
test_loss, test_acc = model.evaluate(test_ds)
print(f"Test accuracy: {test_acc*100:.2f}%")
```

```
C:\Users\user\AppData\Local\Programs\Python\Python311\Lib\site-
packages\keras\src\saving\saving_lib.py:757: UserWarning: Skipping
variable loading for optimizer 'adam', because it has 500 variables
whereas the saved optimizer has 18 variables.
  saveable.load_own_variables(weights_store.get(inner_path))

15/15 ━━━━━━━━━━━━━━━━━━━━ 15s 991ms/step - accuracy: 1.0000 - loss:
0.0294
Test accuracy: 100.00%
```

```
Batch Prediction With Disease Display

for images, labels in test_ds.take(1):
    preds = model.predict(images)
    for i in range(len(images)):
        pred_label = class_names[np.argmax(preds[i])]
```

```
        disease = species_to_disease.get(pred_label, "Unknown")
        print(f"Sample {i+1}: Predicted Species: {pred_label},
Disease: {disease}, Confidence: {np.max(preds[i])*100:.2f}%")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 8s 8s/step
Sample 1: Predicted Species: AEDES, Disease: Dengue, Confidence:
99.94%
Sample 2: Predicted Species: AEDES, Disease: Dengue, Confidence:
99.84%
Sample 3: Predicted Species: AEDES, Disease: Dengue, Confidence:
99.81%
Sample 4: Predicted Species: AEDES, Disease: Dengue, Confidence:
98.33%
Sample 5: Predicted Species: AEDES, Disease: Dengue, Confidence:
97.15%
Sample 6: Predicted Species: AEDES, Disease: Dengue, Confidence:
87.36%
Sample 7: Predicted Species: AEDES, Disease: Dengue, Confidence:
71.29%
Sample 8: Predicted Species: AEDES, Disease: Dengue, Confidence:
99.95%
Sample 9: Predicted Species: AEDES, Disease: Dengue, Confidence:
99.30%
Sample 10: Predicted Species: AEDES, Disease: Dengue, Confidence:
99.87%
Sample 11: Predicted Species: AEDES, Disease: Dengue, Confidence:
99.98%
Sample 12: Predicted Species: AEDES, Disease: Dengue, Confidence:
99.50%
Sample 13: Predicted Species: AEDES, Disease: Dengue, Confidence:
99.87%
Sample 14: Predicted Species: AEDES, Disease: Dengue, Confidence:
98.74%
Sample 15: Predicted Species: AEDES, Disease: Dengue, Confidence:
98.72%
Sample 16: Predicted Species: AEDES, Disease: Dengue, Confidence:
91.54%
Sample 17: Predicted Species: AEDES, Disease: Dengue, Confidence:
99.81%
Sample 18: Predicted Species: AEDES, Disease: Dengue, Confidence:
99.93%
Sample 19: Predicted Species: AEDES, Disease: Dengue, Confidence:
62.52%
Sample 20: Predicted Species: AEDES, Disease: Dengue, Confidence:
99.99%
Sample 21: Predicted Species: AEDES, Disease: Dengue, Confidence:
100.00%
Sample 22: Predicted Species: AEDES, Disease: Dengue, Confidence:
99.92%
Sample 23: Predicted Species: AEDES, Disease: Dengue, Confidence:
```

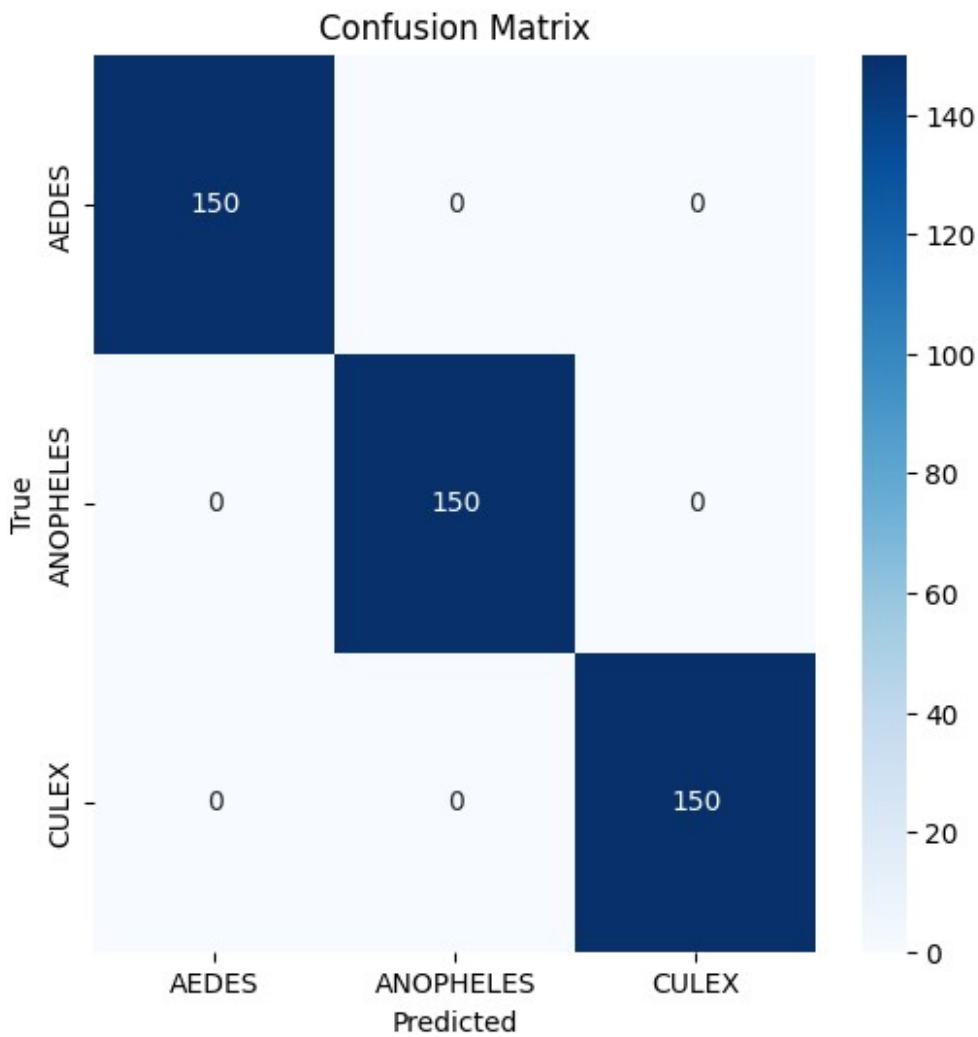
```
96.64%
Sample 24: Predicted Species: AEDES, Disease: Dengue, Confidence:
98.36%
Sample 25: Predicted Species: AEDES, Disease: Dengue, Confidence:
99.59%
Sample 26: Predicted Species: AEDES, Disease: Dengue, Confidence:
97.15%
Sample 27: Predicted Species: AEDES, Disease: Dengue, Confidence:
97.70%
Sample 28: Predicted Species: AEDES, Disease: Dengue, Confidence:
98.45%
Sample 29: Predicted Species: AEDES, Disease: Dengue, Confidence:
97.23%
Sample 30: Predicted Species: AEDES, Disease: Dengue, Confidence:
99.84%
Sample 31: Predicted Species: AEDES, Disease: Dengue, Confidence:
87.91%
Sample 32: Predicted Species: AEDES, Disease: Dengue, Confidence:
99.26%
```

Classification Report and Confusion Matrix

```
from sklearn.metrics import confusion_matrix, classification_report

y_true, y_pred = [], []
for images, labels in test_ds:
    preds = model.predict(images)
    y_true.extend(labels.numpy())
    y_pred.extend(np.argmax(preds, axis=1))

print(classification_report(y_true, y_pred, target_names=class_names))

cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(6,6))
sns.heatmap(cm, annot=True, fmt='d', xticklabels=class_names,
yticklabels=class_names, cmap="Blues")
plt.xlabel("Predicted"); plt.ylabel("True"); plt.title("Confusion
Matrix")
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 999ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 917ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 951ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 902ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 976ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 963ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 945ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 950ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 935ms/step
```

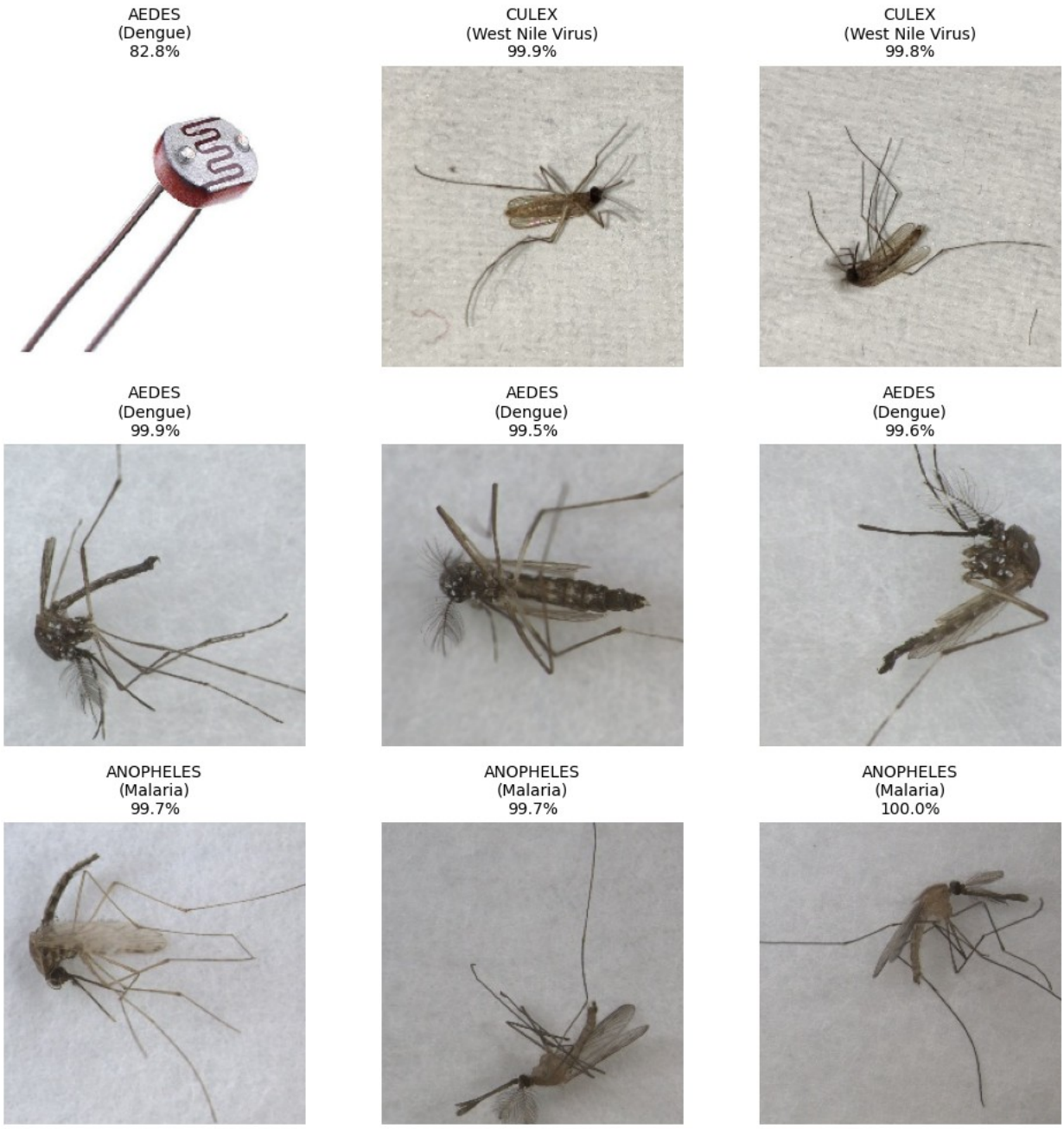
```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 931ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 940ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 992ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 940ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 6s 6s/step
              precision    recall  f1-score   support

       AEDES       1.00      1.00      1.00       150
   ANOPHELES       1.00      1.00      1.00       150
       CULEX       1.00      1.00      1.00       150

    accuracy                           1.00       450
   macro avg       1.00      1.00      1.00       450
weighted avg       1.00      1.00      1.00       450
```




```
Predict a New Image
```

```
from tensorflow.keras.utils import load_img, img_to_array

def predict_images_with_disease(img_paths):
    plt.figure(figsize=(10, 10))
    for i, img_path in enumerate(img_paths[:9]):
        plt.subplot(3, 3, i + 1)
        if not is_image_okay(img_path):
            plt.text(0.5, 0.5, "Invalid Image\nor Not a Mosquito",
                     fontsize=12, ha='center', va='center',
color='red')
            plt.axis("off")
            continue
        img = load_img(img_path, target_size=(IMG_SIZE, IMG_SIZE))
        img_array = img_to_array(img)
        img_array = np.expand_dims(img_array, axis=0)
        img_array =
tf.keras.applications.efficientnet_v2.preprocess_input(img_array)
        pred = model.predict(img_array, verbose=0)
        confidence = np.max(pred) * 100
        pred_label = class_names[np.argmax(pred)]
        disease = species_to_disease.get(pred_label, "Unknown")
        plt.imshow(img)
        if confidence < 60:
            plt.title(f"Low Confidence\n({confidence:.1f}%)",
fontsize=10, color='red')
        else:
            plt.title(f"{pred_label}\n({disease})\n{confidence:.1f}%",
fontsize=10)
        plt.axis("off")
    plt.tight_layout()
    plt.show()

img_paths = [
    r"C:\Users\user\Downloads\ldr-photoresistor-photo-light-sensitive-
resistor-light-dependent-resistor-500x500.jpg",
    r"C:\Users\user\Downloads\AI (Learing Materials)\Mosquito Dataset
for Classification_Diaz G\mosquito_data\Mosquito_dataset\CULEX\
transformed_0799.jpg",
    r"C:\Users\user\Downloads\AI (Learing Materials)\Mosquito Dataset
for Classification_Diaz G\mosquito_data\Mosquito_dataset\CULEX\
transformed_0811.jpg",
    r"C:\Users\user\Downloads\AI (Learing Materials)\Mosquito Dataset
for Classification_Diaz G\mosquito_data\Mosquito_dataset\AEDES\
transformed_0989.jpg",
    r"C:\Users\user\Downloads\AI (Learing Materials)\Mosquito Dataset
for Classification_Diaz G\mosquito_data\Mosquito_dataset\AEDES\
transformed_0995.jpg",
    r"C:\Users\user\Downloads\AI (Learing Materials)\Mosquito Dataset
for Classification_Diaz G\mosquito_data\Mosquito_dataset\AEDES\
transformed_0002.jpg",
```

```
    r"C:\Users\user\Downloads\AI (Learing Materials)\Mosquito Dataset
for Classification_Diaz G\mosquito_data\Mosquito_dataset\ANOPHELES\
transformed_0893.jpg",
    r"C:\Users\user\Downloads\AI (Learing Materials)\Mosquito Dataset
for Classification_Diaz G\mosquito_data\Mosquito_dataset\ANOPHELES\
transformed_0900.jpg",
    r"C:\Users\user\Downloads\AI (Learing Materials)\Mosquito Dataset
for Classification_Diaz G\mosquito_data\Mosquito_dataset\ANOPHELES\
transformed_0903.jpg"
]

predict_images_with_disease(img_paths)
```




```
This part defines the prediction and visualization of the species and
their associated disease for up to
    9 user-specified images in a 3x3 grid.
It also defines the guidelines of the displaying of the output. In
simple words, for each image path:
    - If the image is invalid, it displays an error message in that
grid spot.
    - If valid:
            * The system loads and preprocesses the image for the
model.
```

```
            * The system uses the trained model to predict the
mosquito species and the associated disease.
            * The system shows the image with its predicted species,
disease,
                and the model’s confidence as the subplot title.
    - If the model’s confidence is low, it marks the prediction in red
as a warning.

Since the first image of the grid, which is clearly an LDR who is not
part of the dataset, was still accepted
    and was tried to predict on by the model is because it is a
working image and not a corrupted image or image
    with an invalid format. However, despite being accepted and
labeled, the confidence level of the said image
    is evidently low in comparison to images which is part of the
dataset.

    Moreover,
        - If the image is valid, whether that of a mosquito or not a
mosquito, the system will attemp to classify
          the image, but will be given low confidence level to
indicate that the image is irrelevant to the dataset.
```